

LAB 4 — VIBECODING

Dots and Boxes — Chat History

PROMPT 1

USER PROMPT

hey, i have to finish this VibeCoding lab for Dots and Boxes. i've uploaded the lab handout and the original project files/README.

can you go through the project properly and start with Task 1 from the README? i don't want you to just assume that something is broken, so actually trace the game logic and figure out what happens when a player completes one box and when one move completes two boxes.

check how completed boxes are stored, how they're displayed, how the score is updated and how the turn changes. if the score and extra-turn behaviour is already working, say that clearly and identify the actual issue instead of inventing a bug.

then fix Task 1 while keeping the existing modular structure. don't rewrite the whole project and don't unnecessarily change main.py.

give me the complete code for all files that need to change, not just snippets. and before giving me the code, make sure the changed files are compatible with each other and won't introduce something like a set/dictionary mismatch or another runtime error.

also keep the normal Dots and Boxes rules working: completing a box gives a point and the same player gets another turn. these are the files scenario 23 - Dots and Boxes

Overview

Dots and Boxes is a turn-based grid game. Players draw one line at a time between adjacent dots. Completing the fourth side of a box claims that box, earns a point, and gives the same player another turn.

The starter project is deliberately split into separate modules for game flow, board state, and rule validation. Read the existing implementation carefully before changing it.

How to run

From this folder:

```
python main.py
```

Enter moves in the form:

```
H row column
```

or:

```
V row column
```

Rows and columns start at zero.

Task 1 — Reproduce and investigate the bug

Run several games and deliberately create situations where a move completes one or more boxes.

Compare the score and turn behaviour before and after box completion. Identify the incorrect behaviour, trace it through the board and game-state logic, and fix it without replacing the modular structure.

Your before-change video should capture the broken behaviour clearly.

Task 2 — Add a meaningful feature

Add a substantial gameplay feature that requires changes across more than one module.

The feature should make the game more complete rather than simply changing text or appearance. It should interact correctly with the existing board state and turn/score system.

Task 3 — Validation and robustness

Strengthen input and game-state handling.

The program should safely handle malformed commands, invalid coordinates, repeated lines, and moves made after the board is already complete. Invalid input must not corrupt the board or score.

Task 4 — Testing and quality

Create or expand automated tests covering the important game rules.

Include tests for at least:

- a valid horizontal move
- a valid vertical move
- an invalid/repeated move
- completion of a box
- the end-of-game condition

Document the changes you made and any design decisions that were important to the solution.

Constraints

- Keep the project modular.
- Do not replace the game with an unrelated implementation.
- Preserve the existing gameplay.
- Avoid putting all new logic into `main.py`.
- Keep third-party dependencies out unless there is a clear need for them.

Submission Checklist

Submission is only the following three things:

- A 10-second video of gameplay **before** your changes, showing the bug/broken behavior
- A 10-second video of gameplay **after** your changes, showing the bug fixed and the new features working
- The Chat/LLM used page link, with the complete chat history

CHATGPT RESPONSE 1

I went through the Task 1 requirements in the handout and traced the actual code you supplied. The handout specifically says to **run the game, reproduce the box-completion behaviour, trace it through the board/game-state logic, and then fix the actual issue rather than replacing the modular structure**. [@filecite?turn0file0?L13-L23](#)

Task 1 trace — important finding

With the code you pasted, **Task 1's claimed score/extra-turn bug is already fixed**. I would **not** invent a change just to say we fixed something.

Here is exactly what happens.

1. A line is played

`game.py` does:

```
before = set(self.board.completed)
self.board.add_line(orientation, row, col)
newly_completed = completed_boxes(self.board, before)
```

So it takes a snapshot of completed boxes **before** the move.

2. `Board.add\_line()` updates the board

In `board.py`:

```
self.horizontal[row][col] = True
```

or

```
self.vertical[row][col] = True
```

and then:

```
self._update_completed()
```

`_update_completed()` checks every box and adds its coordinate `(r, c)` to:

```
self.completed
```

So completed boxes are stored as a **set** of `(row, column)` tuples.

3. One box is completed

For example, after the final side of `(0, 0)` is drawn:

```
before = set()
board.completed = {(0, 0)}
```

Then:

```
completed_boxes(board, before)
```

does:

```
len(board.completed - before)
```

which gives:

```
1
```

So the score becomes:

```
self.scores[self.current] += 1
```

And crucially, this code does **not** change `self.current`:

```
if newly_completed:
    self.scores[self.current] += newly_completed
```

Therefore the same player gets another turn.

4. One move completes TWO boxes

This is the important test.

For a 2-column board, suppose the horizontal sides are already present and the final vertical line between the two boxes is played.

Before:

```
before = set()
```

After the move:

```
board.completed = {(0, 0), (0, 1)}
```

Then:

```
board.completed - before
```

is:

```
{(0, 0), (0, 1)}
```

and:

```
len(...) == 2
```

Therefore:

```
self.scores[self.current] += 2
```

and the player **keeps the turn**.

So the supplied implementation correctly handles both:

Situation	<code>completed_boxes()</code>	Score change	Turn
-----------	--------------------------------	--------------	------

---	---	---	---
-----	-----	-----	-----

No box	0	+0	Changes
--------	---	----	---------

One box	1	+1	Same player
---------	---	----	-------------

Two boxes	2	+2	Same player
-----------	---	----	-------------

Completed-box display is also working

``Board.display()`` checks:

```
(r, c) in self.completed
```

and prints:

```
X
```

inside that box.

So the same ``set`` of tuple coordinates is consistently used for:

```
Board storage → completion detection → score calculation → display
```

There is **no set/dictionary mismatch** in the code you provided.

So what is the actual Task 1 issue?

Based strictly on the files/code available to me, **there isn't currently a Task 1 score/extra-turn bug to fix**.

The code you've pasted already contains the logic that Task 1 asks you to arrive at.

I also wouldn't change ``main.py``, because it has nothing to do with this logic.

One thing that IS weak

There is a separate robustness issue in ``Board.add_line()``:

```
def add_line(self, orientation, row, col):
    if orientation == "H":
        self.horizontal[row][col] = True
    else:
        self.vertical[row][col] = True
```

``Board.add_line()`` itself doesn't validate coordinates or repeated moves. **However**, ``game.py`` calls ``valid_move()`` before it reaches ``add_line()``, so normal gameplay is protected.

That belongs much more naturally to **Task 3**, which explicitly asks for malformed commands, invalid coordinates, repeated lines, and completed-game handling. [filecite2turn0file0L18-L23](#)

Therefore, I would NOT give you fake "fixed" code for Task 1

If you submit a change that doesn't fix an actual bug, you risk exactly what your prompt is trying to avoid: claiming that something was broken when it wasn't.

The correct Task 1 conclusion for the code you've supplied is:

> The investigation showed that box completion, score updates, and extra-turn behaviour are already implemented correctly.

``Board.completed`` stores completed boxes as `(row, col)`` tuples in a set. After each move, ``completed_boxes()`` compares the new set with a snapshot of the previous set, allowing one move to correctly award one or multiple boxes. When one or more boxes are completed, the current player receives the corresponding points and retains the turn; otherwise the turn changes. Therefore, no Task 1 code change is required in the supplied version.

But there is one catch

You said you have the **original project files/README**, while in this conversation I currently have the handout plus the code you pasted. The handout itself says the assigned repository/README is what should be used to understand the deliverables.

[filecite2turn0file0L13-L16](#)

If the pasted code is **after someone already made changes**, then we've potentially been given the fixed version rather than the original buggy version.

Send/upload the actual original project files (especially ``board.py``, ``game.py``, ``rules.py``, ``main.py``, and README), or the project ZIP. Then I can compare the real starter implementation and trace the genuine Task 1 bug before changing anything.

PROMPT 2

USER PROMPT

okay, now let's continue with the next requirements from the README.

for Task 2, add a meaningful gameplay feature that actually changes how the game works. let's add custom board size so the players can choose the number of rows and columns when the game starts, like 2x2 or 3x3.

make sure it works properly with the existing board state, box ownership, scoring and turn system.

then do Task 3 as well. the game should safely handle malformed commands, letters instead of numbers, negative or out-of-range coordinates, invalid H/V values, repeated lines and attempts to play after the board is complete.

invalid input shouldn't crash the game or accidentally change the board, score or turn.

keep the existing modular structure and don't rewrite everything. main.py should stay unchanged unless there is absolutely no other way to implement something.

give me the complete updated code for every file that needs to change, and check that all the files still work together before giving me the final code.

CHATGPT RESPONSE 2

Yep. **Task 2 and Task 3 can be implemented cleanly without touching `main.py`.** I kept the original modular structure:

- `board.py` → board dimensions, lines, completed boxes, display, safe line insertion
- `rules.py` → move validation and completed-box calculation
- `game.py` → board-size selection, command parsing, turn/score flow
- `main.py` → **unchanged**

I also tested the interaction end-to-end, including malformed input, repeated moves, scoring, extra turns, and a completed board.

What changed

Task 2 — Custom board size

- At startup:

```
```text
```

Enter board size (rows cols), e.g. 2 2:

```
```
```

- Supports `2 2`, `3 3`, `2 4`, etc.
- `Board` dynamically creates the correct horizontal/vertical arrays.
- Box detection, scoring, display, and game-completion calculation automatically use the selected dimensions.
- `DotsAndBoxes(2, 2)` can also be used directly for testing.

Task 3 — Robust validation

- Bad command formats don't crash.
- Letters in coordinates are rejected.
- Negative coordinates are rejected.
- Out-of-range coordinates are rejected.
- Invalid orientations are rejected.
- Repeated lines are rejected.
- Invalid moves don't change score, turn, or board.
- `play_move()` explicitly refuses moves after the board is complete.
- `Board.add_line()` itself also refuses invalid/repeated lines, providing an additional state-safety layer.

One useful design choice is that **parsing and validation remain separate**: `H 99 0` is syntactically a valid command, but `rules.valid_move()` rejects it because the coordinate is outside the selected board.

`board.py`

```

class Board:
    def __init__(self, rows=2, cols=2):
        if (
            not isinstance(rows, int)
            or not isinstance(cols, int)
            or rows <= 0
            or cols <= 0
        ):
            raise ValueError("Rows and columns must be positive integers.")

        self.rows = rows
        self.cols = cols

        self.horizontal = [[False] * cols for _ in range(rows + 1)]
        self.vertical = [[False] * (cols + 1) for _ in range(rows)]

        # Stores completed boxes as (row, column) tuples.
        self.completed = set()

    def add_line(self, orientation, row, col):
        """
        Add a line only if it is valid and unused.

        Returns True if the board was changed.
        Returns False if the move is invalid or the line already exists.
        """
        orientation = (
            orientation.upper()
            if isinstance(orientation, str)
            else orientation
        )

        if orientation == "H":
            if not (0 <= row <= self.rows and 0 <= col < self.cols):
                return False

            if self.horizontal[row][col]:
                return False

            self.horizontal[row][col] = True

        elif orientation == "V":
            if not (0 <= row < self.rows and 0 <= col <= self.cols):
                return False

            if self.vertical[row][col]:
                return False

            self.vertical[row][col] = True

        else:
            return False

        self._update_completed()
        return True

    def _update_completed(self):
        for r in range(self.rows):
            for c in range(self.cols):
                if (
                    self.horizontal[r][c]
                    and self.horizontal[r + 1][c]
                    and self.vertical[r][c]
                    and self.vertical[r][c + 1]
                )

```

```

        ):
            self.completed.add((r, c))

def is_complete(self):
    total = (
        self.rows * (self.cols + 1)
        + self.cols * (self.rows + 1)
    )

    used = (
        sum(map(sum, self.horizontal))
        + sum(map(sum, self.vertical))
    )

    return used == total

def display(self, scores, current):
    print()
    print(f"Board: {self.rows}x{self.cols}")
    print(
        f"Scores: P1={scores[0]}  "
        f"P2={scores[1]} | Turn: P{current + 1}"
    )

    for r in range(self.rows + 1):
        print(
            ".".join(
                "---" if self.horizontal[r][c] else "  "
                for c in range(self.cols)
            )
        )

        if r < self.rows:
            middle = []

            for c in range(self.cols + 1):
                wall = "|" if self.vertical[r][c] else " "
                middle.append(wall)

                if c < self.cols:
                    box = "X" if (r, c) in self.completed else " "
                    middle.append(" " + box + " ")

            print("".join(middle))

    print()

```

`rules.py`

```

def valid_move(board, orientation, row, col):
    if not isinstance(orientation, str):
        return False

    orientation = orientation.upper()

    if orientation not in {"H", "V"}:
        return False

    if not isinstance(row, int) or not isinstance(col, int):
        return False

    if orientation == "H":
        return (

```

```

        0 <= row <= board.rows
        and 0 <= col < board.cols
        and not board.horizontal[row][col]
    )

    return (
        0 <= row < board.rows
        and 0 <= col <= board.cols
        and not board.vertical[row][col]
    )

def completed_boxes(board, before):
    return len(board.completed - before)

```

`game.py`

```

from board import Board
from rules import valid_move, completed_boxes

class DotsAndBoxes:
    def __init__(self, rows=None, cols=None):
        self.rows = rows
        self.cols = cols
        self.board = None

        self.current = 0
        self.scores = [0, 0]

    def _choose_board_size(self):
        """
        Ask the players for the board dimensions.

        If rows and cols were supplied when constructing the game,
        those values are used instead.
        """
        if self.rows is not None and self.cols is not None:
            if (
                not isinstance(self.rows, int)
                or not isinstance(self.cols, int)
            ):
                raise ValueError(
                    "Rows and columns must be integers."
                )

            if self.rows <= 0 or self.cols <= 0:
                raise ValueError(
                    "Rows and columns must be positive integers."
                )

            return

        while True:
            raw = input(
                "Enter board size (rows cols), e.g. 2 2: "
            ).strip()

            parts = raw.split()

            if len(parts) != 2:
                print(
                    "Invalid format. Enter two positive integers, "

```

```

        "e.g. 2 2."
    )
    continue

    row_text, col_text = parts

    if not row_text.isdigit() or not col_text.isdigit():
        print(
            "Rows and columns must be positive integers."
        )
        continue

    rows = int(row_text)
    cols = int(col_text)

    if rows <= 0 or cols <= 0:
        print(
            "Rows and columns must be greater than zero."
        )
        continue

    self.rows = rows
    self.cols = cols

    return

def _parse_move(self, raw):
    """
    Parse a command such as:
        H 0 1
        V 1 2

    Returns:
        (orientation, row, col)
    or:
        None for malformed input.
    """
    parts = raw.strip().upper().split()

    if len(parts) != 3:
        return None

    orientation, row_text, col_text = parts

    if orientation not in {"H", "V"}:
        return None

    if not row_text.isdigit() or not col_text.isdigit():
        return None

    return (
        orientation,
        int(row_text),
        int(col_text),
    )

def play_move(self, orientation, row, col):
    """
    Attempt to play one move.

    Returns:
        (success, number_of_new_boxes)

    Invalid moves never modify the board, score, or turn.

```

```

"""
# Do not allow moves after the board is complete.
if self.board is None or self.board.is_complete():
    return False, 0

# Validate the move before changing the board.
if not valid_move(
    self.board,
    orientation,
    row,
    col
):
    return False, 0

# Save completed boxes before the move.
before = set(self.board.completed)

# Board performs a second safety check.
if not self.board.add_line(
    orientation,
    row,
    col
):
    return False, 0

# Determine how many boxes this move newly completed.
newly_completed = completed_boxes(
    self.board,
    before
)

if newly_completed:
    # Completing one or more boxes gives points.
    self.scores[self.current] += newly_completed

    # IMPORTANT:
    # The player keeps the turn.
else:
    # No box was completed, so turn changes.
    self.current = 1 - self.current

return True, newly_completed

def run(self):
    self._choose_board_size()

    self.board = Board(
        self.rows,
        self.cols
    )

    self.current = 0
    self.scores = [0, 0]

    print("Dots and Boxes")
    print(
        f"Starting board: "
        f"{self.rows}x{self.cols}"
    )
    print(
        "Enter moves as H row col or V row col."
    )
    print("Rows and columns start at 0.")
    print("Example: H 0 1")

```

```

while not self.board.is_complete():
    self.board.display(
        self.scores,
        self.current
    )

    raw = input(
        f"Player {self.current + 1}, move: "
    ).strip()

    move = self._parse_move(raw)

    if move is None:
        print(
            "Invalid move. Use H/V followed "
            "by numeric row and column."
        )
        continue

    orientation, row, col = move

    success, newly_completed = self.play_move(
        orientation,
        row,
        col
    )

    if not success:
        print(
            "Invalid or already-used move."
        )
        continue

    if newly_completed:
        print(
            f"Player {self.current + 1} "
            f"completed {newly_completed} "
            f"box(es) and plays again."
        )

    self.board.display(
        self.scores,
        self.current
    )

    print("Game over!")

    if self.scores[0] == self.scores[1]:
        print("The game is a draw.")
    else:
        winner = (
            1
            if self.scores[0] > self.scores[1]
            else 2
        )

        print(
            f"Player {winner} wins!"
        )

```

`main.py`

****No change. Keep your existing file exactly as it is:****

```
from game import DotsAndBoxes

if __name__ == "__main__":
    DotsAndBoxes().run()
```

What I verified

I checked the modules together rather than checking each file in isolation:

- ``python -m py_compile board.py rules.py game.py main.py`` passes.
- ``2x2``, ``3x3``, etc. are represented correctly by the board arrays.
- A normal move changes the correct line.
- A repeated line is rejected.
- ``H`` and ``V`` boundaries are different and correctly validated.
- Negative/out-of-range coordinates are rejected.
- Malformed commands don't crash the game.
- Completing **one box** gives 1 point and retains the turn.
- A move completing **two boxes** gives 2 points and retains the turn.
- A move completing no boxes changes the turn.
- Once the board is complete, ``play_move()`` refuses another move without modifying anything.
- The existing ``completed`` representation remains a **set of ``(row, col)`` tuples**, so there is no set/dictionary compatibility problem.

For your Task 3 explanation, the important design point is that **validation happens before mutation**. That means an invalid command cannot accidentally reach the code that changes the board, and ``Board.add_line()`` has its own safety check as a second layer.

PROMPT 3

USER PROMPT

alright, now let's finish the lab with Task 4.

create a test_dots_and_boxes.py using Python's built-in unittest.

make sure the tests cover the important rules from the README: a valid horizontal move, a valid vertical move, an invalid or repeated move, completing a box and the end-of-game condition. also test input parsing or validation if it makes sense.

after creating the tests, go through the whole solution once and check that Tasks 1, 2, 3 and 4 are actually covered and that the project is still modular.

also check that board.py, game.py, rules.py and the test file are all compatible with each other and that the ownership tracking, scoring, extra turns, custom board size and invalid-input handling all work together.

if anything needs fixing, give me the complete corrected files.

finally, tell me the exact command i should run to test everything and what i should manually check when i run python main.py before submitting.

CHATGPT RESPONSE 3

Done. I ran the complete `unittest` suite against the Task 2 + Task 3 implementation, and after correcting one **test expectation** (not a game bug), all **10 tests pass**.

Test file

I created:

[Download `test\_dots\_and\_boxes.py`](sandbox:/mnt/data/dots_and_boxes_task4/test_dots_and_boxes.py)

It covers:

| Requirement | Test |

|---|---|

| Valid horizontal move |  |

| Valid vertical move |  |

| Invalid orientation/coordinates |  |

| Repeated line |  |

| One box completion |  |

| Score update |  |

| Extra turn after completing box |  |

| Two boxes completed by one move |  |

| Custom board dimensions |  |

| Input parsing |  |

| Invalid move doesn't mutate state |  |

| End-of-game condition |  |

| Move after game completion |  |

Important thing the tests caught

The implementation's turn logic is correct.

For a 1×1 board:

```
P1 → first side
P2 → second side
P1 → third side
P2 → fourth side → completes box
```

Therefore P2 gets the point and keeps the turn:

```
Scores: P1=0 P2=1
Turn: P2
```

That is normal Dots and Boxes behaviour. I initially wrote the test expecting P1 to score, and the test correctly caught **my** test error******. I changed the test, not the game logic.

Final Task 1–4 status

Task 1 — Bug investigation

****Covered.****

The supplied implementation already correctly handles:

- completed-box tracking
- one-box completion
- multiple boxes completed by one move
- score updates
- extra turns

We therefore didn't fabricate a Task 1 bug.

Task 2 — Custom board size

****Covered.****

Players can enter:

```
2 2
```

or:

```
3 3
```

or other positive dimensions.

The board dynamically creates:

```
horizontal → (rows + 1) × cols
vertical   → rows × (cols + 1)
```

and box detection/scoring continues to use the same board state.

Task 3 — Robustness

****Covered.****

Rejected safely:

```
hello
H
H 1
H a 2
X 0 0
H -1 0
H 999 999
V 999 0
```

Repeated lines are rejected, and invalid moves don't alter:

- board
- completed boxes
- score
- current player

A completed board also refuses further moves.

Task 4 — Automated testing

****Covered.****

The final run was:

```
Ran 10 tests in 0.001s
OK
```

So the four modules are compatible with each other.

Exact command to test everything

Put ``test_dots_and_boxes.py`` in the **same folder** as:

```
board.py
game.py
rules.py
main.py
```

Then from that folder run:

```
python -m unittest discover -v
```

You should get:

```
Ran 10 tests
OK
```

You can also run the test file directly:

```
python -m unittest test_dots_and_boxes -v
```

I recommend the **discover** command for your submission because it demonstrates that Python can discover and execute the test suite.

What to manually check with ``python main.py``

Run:

```
python main.py
```

1. Custom board

First test:

```
Enter board size (rows cols), e.g. 2 2: 2 2
```

Make sure the displayed board is 2×2.

Then try another run with:

```
3 3
```

and make sure the larger board appears.

2. Invalid input

Try:

```
hello
```

Then:

```
H
```

Then:

```
H a 1
```

Then:

```
X 0 0
```

Then:

```
H -1 0
```

The game should **not crash**.

3. Repeated line

Play:

```
H 0 0
```

Then try:

```
H 0 0
```

The second attempt should say:

```
Invalid or already-used move.
```

and **the turn should not change** because an invalid move isn't a move.

4. Complete a box

On a 1×1 board, you can make this particularly easy to demonstrate:

```
1 1
```

Then play four sides:

```
H 0 0
H 1 0
V 0 0
V 0 1
```

The final move should:

- display `X` inside the box
- give the current player +1
- keep the same player's turn

5. Two boxes with one move

Use:

```
1 2
```

Build the two boxes so that the **shared vertical line** is the final missing line.

That move should produce:

```
completed 2 box(es)
```

and award **2 points** to the same player while retaining their turn.

6. Finish the entire game

Make sure that after the final available line:

```
Game over!
```

appears, and the program doesn't accept another move.

One thing I would NOT change

Keep `main.py` exactly as it is:

```
from game import DotsAndBoxes

if __name__ == "__main__":
    DotsAndBoxes().run()
```

There is no reason to put Task 2/3/4 logic there. The current separation is much cleaner:

```
main.py
  ↓
game.py
  ├── board.py    → board state
  ├── rules.py    → rule validation
  ↑
test_dots_and_boxes.py
```

That directly satisfies the README's requirement to **keep the project modular and avoid putting all the new logic into `main.py`**. [filecite turns file 02L18-L23](#)

So at this point, your **code-side Tasks 1–4 are covered**, and the final test suite passes.